-- 以下为AI生成的图文笔记内容 --

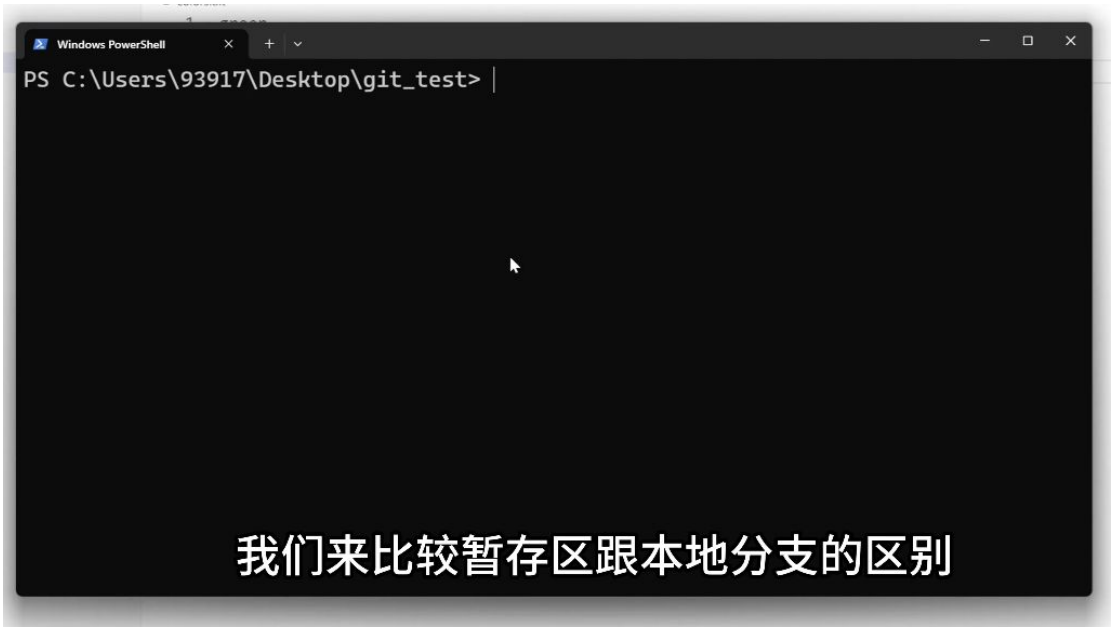
一、*git diff命令00:00*

1. git diff基础用法 00:03

-

- **基本功能**：用于比较Git中不同版本或状态之间的差异
- **默认行为**：不加参数时比较工作目录和暂存区的差异
- **示例操作**：
  - 修改文件后直接运行git diff查看未暂存的更改
  - 添加文件到暂存区后再次运行git diff将不再显示该文件的差异

1）比较暂存区与本地分支 00:35

- 

- **命令格式**：git diff --staged
- **比较对象**：显示暂存区与最后一次提交(本地分支)的差异
- **应用场景**：确认即将提交的内容是否正确

2）比较工作目录与本地分支 01:02

- **命令格式**：git diff HEAD
- **比较对象**：显示工作目录与本地分支最新提交的差异
- **特点**：包含所有未暂存和已暂存的修改

2. 分支比较 01:35

1）两点比较法 02:21

-

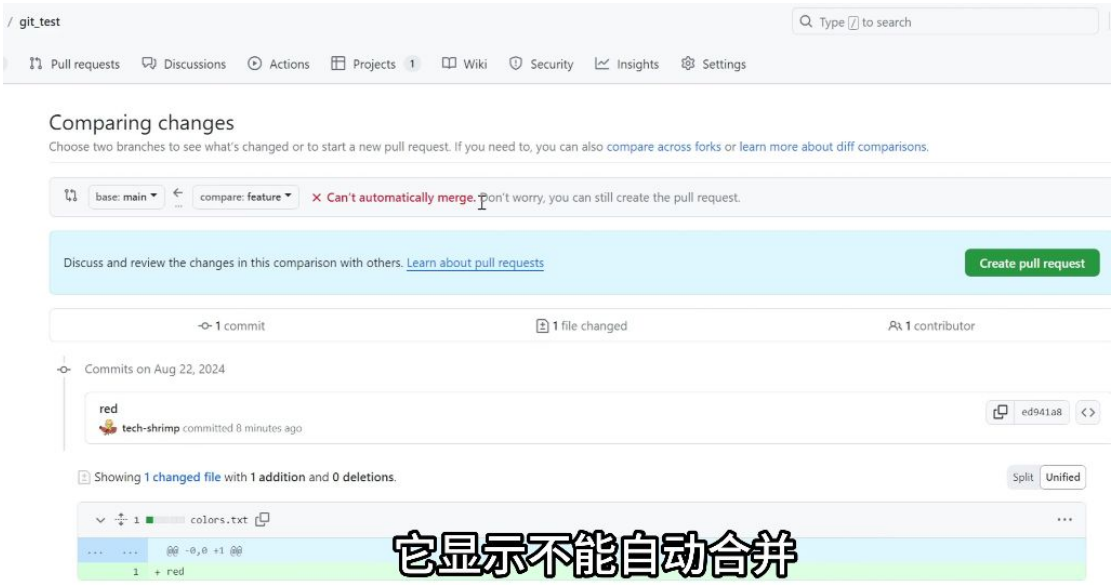
- **命令格式**：git diff branch1..branch2
- **比较原理**：直接比较两个分支最新提交的绝对差异
- **示例**：比较main分支和feature分支的差异
- **特点**：
  - 显示两个分支分叉后的所有差异
  - 顺序影响显示结果（前后分支位置可互换）

2）三点比较法

-

- **命令格式**：git diff branch1...branch2
- **比较原理**：
  - 先找到两个分支的共同祖先提交
  - 将后面指定的分支与共同祖先比较
- **特点**：
  - GitHub默认使用三点比较
  - 只显示后面分支独有的修改

3. 冲突处理 04:38



-
- **处理流程**：
  - 本地切换到feature分支：git switch feature
  - 合并main分支：git merge main
  - 解决冲突后提交：git commit -m "resolve conflict"
  - 推送修改：git push
  - 再创建PR
- **注意事项**：
  - 必须在本地先解决冲突再创建PR
  - 强制推送(git push -f)应谨慎使用

4. 高级比较技巧 05:36

1）比较特定提交

- **命令格式**：git diff commit1 commit2
- **应用场景**：比较任意两个历史提交之间的差异

2）标签比较

- **比较方式**：
  - 标签间比较：git diff v1.0 v1.1

- 标签与分支比较：git diff v1.0 main
- 标签与提交比较：git diff v1.0 commitID

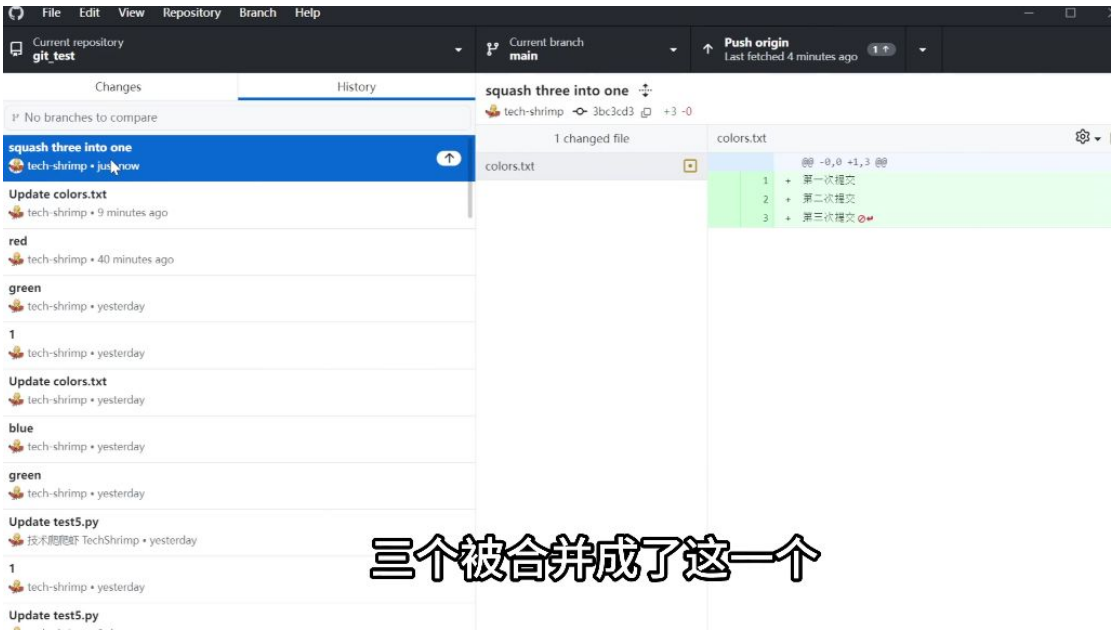
5. 代码暂存与恢复 06:10

1）git stash基本用法

- **存储修改**：
  - 常规方法：先git add再git stash
  - 快捷方法：git stash -a（包含未跟踪文件）
- **查看存储列表**：git stash list
- **恢复修改**：
  - 保留存储：git stash apply stash@{0}
  - 删除存储：git stash drop stash@{0}
  - 恢复并删除：git stash pop stash@{0}

6. 提交历史修改 10:11

1）合并多个提交(squash)



- **reset方法**：
  - git reset --mixed commitID（回退到合并起点）
  - 重新提交：git add . + git commit -m "message"

- **rebase方法**：
  - git rebase -i commitID
  - 将需要合并的提交标记为"squash"
  - 编辑最终提交信息

2）注意事项

- **强制推送**：修改已推送的历史后必须使用git push -f
- **协作分支**：禁止在多人协作的分支上修改历史
- **交互式rebase**：可以修改任意历史提交（pick/reword/edit等）

二、知识小结

知识点	核心内容	易混淆点/重点	难度系数
git diff基础用法	比较工作目录与暂存区差异：git diff 比较暂存区与本地分支：git diff --staged 比较工作目录与本地分支：git diff HEAD	**三种比较模式的区别**： - 无参数：工作区↔暂存区 - --staged：暂存区↔分支 - HEAD：工作区↔分支	★★
分支差异比较	两点比较：git diff branchA..branchB 三点比较：git diff branchA...branchB	**关键区别**： - 两点比较：直接对比分支末端 - 三点比较：对比共同祖先与后者分支	★★★
冲突解决流程	1. 本地合并git merge 2. 手动解决冲突文件 3. git add标记解决 4. 提交并推送	**必须注意**： - 需在特性分支合并主分支 - 解决后重新提交PR	★★★★
stash操作	存储修改：git stash -a（包含未暂存文件）	**模式差异**： - apply保留记录	★★

	恢复修改：git stash apply/pop 删除记录：git stash drop	- pop自动删除记录	
tag管理	创建标签：git tag v1.0.0 [commitID] 推送标签：git push --tags 删除标签：git tag -d+ git push --delete	**对比功能**： 标签可与分支/commit互相对比	★★
提交压缩	方法1：git reset --mixed+ 重新提交 方法2：交互式rebase（git rebase -i）	**危险操作**： - 已推送历史需强制推送 - 禁止在共享分支使用	★★★★
rebase高级应用	交互式修改历史：pick/reword/squash/edit等指令	**限制条件**： 必须保留至少一个pick提交	★★★★